

Faculty of Computer Technology and Cybersecurity
Department of Computer Engineering
ARCHITECTURE AND ORGANIZATION OF COMPUTER SYSTEMS
WORKSHEET - 4

Full Name: Daniil Kalts

Group: IT2-2404SE

Q1: How do we design a 4-bit CLA?

A 4-bit carry look-ahead adder (CLA) adds two 4-bit numbers. For each bit i , it first finds $P_i = A_i \text{ XOR } B_i$ (propagate) and $G_i = A_i \text{ AND } B_i$ (generate). P_i means the bit can pass an incoming carry; G_i means it creates a carry by itself.

Let C_0 be the carry entering the 4-bit adder. The CLA calculates the next carries directly:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

Here $+$ means OR and adjacent symbols mean AND. Finally, each sum bit is $S_i = P_i \text{ XOR } C_i$. Because the carries are calculated from P and G together, we do not wait for a carry to pass through all four full adders.

Q2: What hardware is needed for a 4-bit block CLA, and what is the total delay?

One 4-bit block needs four PG units (an XOR and an AND gate per bit), a CLA network for its carries, and XOR gates for its four sum bits. To connect blocks in a larger adder, it also produces a block propagate signal and a block generate signal. These two signals tell the next level whether the entire block passes or creates a carry.

In the lecture's simplified model, a 4-bit CLA takes about 6Δ : 2Δ to form the bit P and G signals, 2Δ to calculate the carries, and 2Δ for the sums. Δ is a unit of logic delay used for comparison, not a fixed time in seconds.

For a 16-bit adder built from four 4-bit blocks, simply cascading the blocks gives $4 \times 6\Delta = 24\Delta$. Adding a block-level CLA to calculate the carries between blocks in parallel gives about 12Δ in the lecture's model.

Q3: What is a Super Block CLA? Define its propagate and generate signals.

A Super Block CLA is another level of carry look-ahead. In the 64-bit design, each 16-bit unit is treated as one stage. The super-block logic uses the propagate and generate signals from these stages to calculate carries at bit positions 16, 32, 48 and 64 in parallel.

For super block 0, consider its four smaller blocks numbered 0 to 3. Let PB_0 to PB_3 mean their block propagate signals, and GB_0 to GB_3 mean their block generate signals. The signals for the whole super block are:

$$PSB_0 = PB_3PB_2PB_1PB_0$$

$$GSB_0 = GB_3 + PB_3GB_2 + PB_3PB_2GB_1 + PB_3PB_2PB_1GB_0$$

PSB_0 is 1 only when all four smaller blocks pass an incoming carry. GSB_0 is 1 when a carry is created inside one of them and reaches the output. Thus the outgoing carry is $C_{16} = GSB_0 + PSB_0C_0$. The lecture estimates about 20Δ for the complete 64-bit hierarchical CLA, versus 128Δ for a 64-bit ripple-carry adder.

Q4: How does an ALU perform subtraction?

The ALU can subtract using its adder. In fixed-width binary arithmetic, $A - B = A + (\text{NOT } B) + 1$. NOT B flips each bit of B; adding 1 gives the two's complement of B.

A shared add/subtract circuit uses one control bit called SUB. When $SUB = 0$, it sends B unchanged into the adder and sets carry-in to 0, so the result is $A + B$. When $SUB = 1$, it flips every bit of B and sets carry-in to 1, so the result is $A - B$. The same adder hardware works for both operations.

For example, using 4 bits: $5 - 3$ uses $0101 + 1100 + 1$. This gives $1\ 0010$; the extra carry is discarded, leaving the 4-bit result 0010 (2). The ALU may also set flags such as zero, carry/borrow and signed overflow; their exact meaning depends on the processor.

Q5: What is a register? Discuss its kinds.

A register is a small, fast storage location inside the CPU. It holds a value that the processor needs right now, such as data, an address or part of the current instruction. A register usually holds one machine word.

- **General-purpose registers:** hold numbers, addresses and intermediate results for programs.
- **Data or accumulator registers:** hold operands and results while arithmetic or logical operations are performed.
- **Address registers:** hold memory addresses; the stack pointer (SP) points to the current top of the stack.
- **Control registers:** help run instructions. The program counter (PC) holds the address of the next instruction, and the instruction register (IR) holds the current instruction.
- **Status or flag register:** records conditions such as zero result, carry and overflow.